

Interdisciplinary Master of Science in
«Artificial Intelligence»

SUBJECT: DEEP LEARNING

TITLE: DEEPFAKE IMAGE DETECTION
USING DEEP LEARNING METHODS

APOSTOLOS STAVROU MTN2526

SUPERVISORS:

THEODOROS GIANNAKOPOULOS

IOANNIS KAKOGEORGIOU

1. INTRODUCTION	3
2. DATASET	3
2.1 PRIMARY DATASET.....	3
2.2 CROSS-DISTRIBUTION DATASET	4
3. EXPERIMENTAL SETUP	4
3.1 EVALUATION METRICS.....	4
3.2 TRAIN / VALIDATION / TEST PROTOCOL.....	4
3.3 FRAMEWORK AND HARDWARE	5
4. STAGE 1: BASELINE FC NETWORK ON HAND-CRAFTED FEATURES	5
4.1 FEATURE EXTRACTION	5
4.2 MODEL ARCHITECTURE.....	5
4.3 TRAINING.....	6
4.4 RESULTS	6
5. STAGE 2: CUSTOM CNN ARCHITECTURES	7
5.1 DATA PIPELINE.....	7
6. STAGE 3: TRANSFER LEARNING WITH PRETRAINED MODELS	9
6.1 MOTIVATION AND STRATEGY	9
6.2 INPUT AND NORMALISATION.....	10
6.3 AUGMENTATION.....	10
6.4 TRAINING CONFIGURATION	10
6.5 MODEL ARCHITECTURES	11
6.6 RESULTS AND ANALYSIS	11
7. FINAL TEST SET EVALUATION	11
7.1 STAGE PROGRESSION.....	12
7.2 NOTABLE RESULTS	12
8. CROSS-DISTRIBUTION GENERALISATION.....	13
8.1 INTERPRETATION.....	13
9. DISCUSSION.....	14
10. CONCLUSION	15
APPENDIX:	16

1. Introduction

Deepfake detection (the task of classifying images as real or synthetically generated) has become an increasingly relevant problem as generative models grow more sophisticated. Images produced by Generative Adversarial Networks (GANs) can be visually indistinguishable from real photographs, yet they carry subtle artifacts in their frequency structure, texture, and pixel-level statistics that can be identified by a trained classifier.

This project builds a deepfake image detection system using Deep Learning, progressing through three stages of increasing architectural complexity. The design is deliberately incremental: each stage introduces one new component while keeping everything else constant, making it possible to attribute performance differences to specific decisions rather than to confounded variables.

Stage 1 establishes a neural network baseline by extracting hand-crafted image features and feeding them into a Fully Connected (FC) network. This stage tests how well a neural classifier performs on fixed, manually engineered representations before moving to end-to-end learning.

Stage 2 introduces Convolutional Neural Networks (CNNs) trained from scratch on raw image pixels. Three architectures (CNN-A, CNN-B, and CNN-C) explore the impact of network depth, pooling strategy, and data augmentation in a controlled, step-by-step manner.

Stage 3 fine-tunes pretrained ImageNet models (ResNet-18 and EfficientNet-B0) to assess the value of transfer learning for this specific task and establish an upper bound on within-distribution performance.

A final cross-distribution experiment applies all trained models to an out-of-distribution dataset of Stable Diffusion-generated images, testing whether the models learned general deepfake artifacts or patterns specific to GAN-based generation. Throughout the project, the emphasis is on clean experimental design: one variable changes at a time, training conditions are held consistent across models, and the test set is withheld until all models are finalized.

2. Dataset

2.1 Primary Dataset

The primary dataset consists of GAN-generated fake faces paired with real face images. The dataset was pre-split and organized into six folders by split and class:

- train-real / train-fake: 15,000 images each (30,000 total)

- valid-real / valid-fake: 2,500 images each (5,000 total)
- test-real / test-fake: 2,500 images each (5,000 total)

The dataset is perfectly balanced across both classes in every split, eliminating class imbalance as a confounding variable. All images are JPEG format. Images are resized to 128*128 pixels for Stages 1 and 2, and to 224*224 for Stage 3 to match the input requirements of pretrained ImageNet models.

2.2 Cross-Distribution Dataset

A secondary dataset of Stable Diffusion-generated images is used exclusively for the generalisation experiment in Section 8. It consists of two folders (man and woman), each containing 1,500 images, for a total of 3,000 synthetically generated images. All images are fake; there are no real images in this set. Stable Diffusion uses a fundamentally different generation process (diffusion-based rather than adversarial), producing images with a different artifact profile from the primary dataset's GAN images. No model is trained on this data; it serves only as an out-of-distribution evaluation.

3. Experimental Setup

3.1 Evaluation Metrics

All models are evaluated using a consistent suite of metrics: Accuracy, Precision, Recall, F1-Score, ROC-AUC, and Confusion Matrix. Binary labels assign Real = 1 (positive class) and Fake = 0 (negative class). ROC-AUC serves as the primary ranking metric, as it is threshold-independent and robust to balanced class distributions. Confusion matrices are provided for interpretability, labelling True Positives as correctly identified real images and True Negatives as correctly identified fake images.

3.2 Train / Validation / Test Protocol

The validation set is used throughout development for architecture decisions, hyperparameter tuning, and checkpoint selection. The test set is strictly held out and evaluated exactly once (after all models across all three stages are finalized) to provide an unbiased estimate of generalisation performance. Evaluating on the test set during development, even purely for observation, risks unconsciously biasing subsequent decisions toward what performs well on that specific split, which invalidates it as a neutral final estimate. Checkpoints are selected by minimum validation loss rather than accuracy. Loss is a smoother, more sensitive signal that better reflects the learning trajectory when accuracy plateaus, and avoids selecting checkpoints that happen to round well on the accuracy metric while the model is still improving. Best-checkpoint weights are always restored before evaluation.

3.3 Framework and Hardware

All experiments are implemented in PyTorch and executed in Jupyter notebooks on a local GPU-enabled machine. Training custom CNNs from scratch required approximately one hour per model. Fine-tuning pretrained models required approximately 80 minutes each.

4. Stage 1: Baseline FC Network on Hand-Crafted Features

4.1 Feature Extraction

Four descriptors are extracted from each grayscale image and saved to disk as compressed .npz files (a deliberate design choice to avoid re-running the expensive extraction step across experiments). The four descriptors and their dimensions are:

LBP (Local Binary Pattern, 59-dim): Samples 8 neighbours around a circle of radius 2 using the `nri_uniform` method, producing exactly 59 uniform rotation-invariant patterns. Encodes local texture structure and micro-texture variation.

FFT Magnitude Spectrum (64-dim): Computes the 2D Fourier transform, applies `fftshift` and log-magnitude scaling to the full spectrum, then summarises into 64 radial frequency bins by averaging energy at each distance from the centre. GAN-generated images often exhibit unnatural high-frequency patterns (characteristic of the adversarial training process) that are invisible to the eye but detectable in the frequency domain.

DCT Coefficients (64-dim): Retains the top-left 8*8 block of 64 coefficients from the 2D Discrete Cosine Transform, representing the low-to-mid frequency components that carry the most discriminative information. High-frequency DCT coefficients are discarded as noise.

Wavelet Statistics (14-dim): Applies a Daubechies-1 (db1) wavelet decomposition at level 2, producing 7 frequency subbands. Mean and standard deviation are extracted from each subband, yielding a 14-dimensional vector capturing multi-scale texture information.

The four descriptors are concatenated into a single 201-dimensional feature vector per image. A `StandardScaler` is fitted on the training split only and applied consistently to validation and test splits, ensuring that normalisation statistics are not influenced by the distributions being evaluated.

4.2 Model Architecture

The FC network (FCNet) is a three-layer MLP with hidden sizes of 512 and 128:

Input(201) -> Linear(512) -> BatchNorm -> ReLU -> Dropout -> Linear(128) -> BatchNorm -> ReLU -> Dropout -> Linear(1)

The output is a single logit, with sigmoid applied internally by the loss function. Two design decisions are worth noting.

BatchNorm before ReLU: The feature vector is a concatenation of four heterogeneous descriptors occupying very different numerical ranges. Even after StandardScaler normalisation, internal activations can develop mismatched scales across layers. BatchNorm re-normalises per-batch activations at each hidden layer, stabilising training independently of the input scaler.

BCEWithLogitsLoss (single logit, no sigmoid in the forward pass): Fusing sigmoid and binary cross-entropy in a single numerically stable operation avoids gradient underflow when predictions are very confident. The alternative (explicit sigmoid followed by BCELoss) is prone to near-zero and near-one floating-point saturation.

The network has 170,497 trainable parameters.

4.3 Training

Loss: BCEWithLogitsLoss. Optimizer: Adam (lr=0.01). Batch size: 256. Epochs: 30. Dropout: 0.4.

4.4 Results

Metric	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Value	81.54%	82.14%	80.60%	81.36%	0.90

Table 1. FC Network validation results.

The FC baseline achieves 81.54% accuracy and 0.90 ROC-AUC, confirming that the 201-dimensional feature vector carries real discriminative signal. Training accuracy reached approximately 86% while validation settled around 81.5%, indicating mild overfitting consistent with a model that has memorised feature-level patterns without finding a clean general decision boundary.

The more important observation is that the ceiling is imposed by the features themselves. The four descriptors encode specific statistics (texture histograms, frequency magnitudes, wavelet coefficients) but discard all spatial information: where an artifact appears in the image, how it relates to neighbouring regions, and what the local context looks like. These are precisely the signals that distinguish GAN-generated faces from real ones, and no classifier operating on these fixed representations can recover them. This limitation directly motivates the move to convolutional architectures in Stage 2.

5. Stage 2: Custom CNN Architectures

5.1 Data Pipeline

Unlike Stage 1, CNNs receive raw image pixels directly. A custom dataset class loads images from disk on demand, converting each to RGB and applying a transform pipeline. Per-channel mean and standard deviation are computed over the training set in a single pass and saved to disk, serving the same role as Stage 1's StandardScaler: fitted on train only and applied consistently across all splits.

The base transform pipeline (used by CNN-A and CNN-B) consists of: resize to 128*128, ToTensor (scaling pixel values from [0,255] to [0,1]), and Normalize using the computed training statistics. No augmentation is applied in the base pipeline, which is intentional: CNN-A and CNN-B are evaluated on clean, unperturbed inputs so that any performance differences between them are attributable solely to architecture.

5.2 CNN-A: Simple Architecture with Flatten Head

CNN-A is the simplest possible CNN baseline, designed to answer one question: does learning spatial features directly from raw pixels outperform the hand-crafted feature pipeline from Stage 1? The architecture consists of three convolutional blocks followed by a flatten classifier:

- Block 1: Conv2d(3, 32, k=3, pad=1) -> BN -> ReLU -> MaxPool(2) | 128*128 -> 64*64
- Block 2: Conv2d(32, 64, k=3, pad=1) -> BN -> ReLU -> MaxPool(2) | 64*64 -> 32*32
- Block 3: Conv2d(64, 128, k=3, pad=1) -> BN -> ReLU -> MaxPool(2) | 32*32 -> 16*16
- Classifier: Flatten(32,768) -> Dropout -> Linear(256) -> ReLU -> Dropout -> Linear(1)

The filter count doubles at each block (32 -> 64 -> 128) while spatial dimensions halve at each MaxPool. This is a standard CNN design principle: as spatial resolution decreases, feature depth increases, preserving total information capacity across the network. The flatten head feeds the full 32,768-dimensional spatial map into a dense classifier. This large FC layer (approximately 8M parameters) is deliberately included to contrast with the Global Average Pooling approach in CNN-B. CNN-A has 8,482,817 trainable parameters in total.

5.3 CNN-B: Deeper Architecture with Global Average Pooling

CNN-B introduces two deliberate changes from CNN-A: a fourth convolutional block and a Global Average Pooling (GAP) classifier head.

The fourth block (Conv2d(128, 256) -> BN -> ReLU -> MaxPool) reduces spatial dimensions to 8*8 while expanding feature depth to 256. AdaptiveAvgPool2d(1) then averages each of the 256 feature maps to a single scalar, producing a compact 256-dimensional vector fed directly into a Linear(1) output.

GAP is a significant architectural shift. The flatten head in CNN-A requires approximately 8M parameters in its classifier alone and must memorise which spatial positions are discriminative. GAP reduces the classifier to 256 parameters and forces the network to produce globally meaningful feature maps, responding to whether an artifact exists anywhere in the image rather than at a memorised location. This is particularly well-suited to deepfake detection, where artifacts have no fixed spatial location. GAP was introduced in the Network in Network architecture and later popularised by GoogLeNet, and is now standard in modern CNN classifiers.

CNN-B achieves this with 389,633 trainable parameters (22 times fewer than CNN-A) while being architecturally deeper.

5.4 CNN-C: CNN-B with Data Augmentation

CNN-C is architecturally identical to CNN-B. The only change is the training data pipeline, which introduces mild augmentation:

- RandomHorizontalFlip (p=0.5): makes the model invariant to horizontal pose variation
- ColorJitter (brightness=0.2, contrast=0.2, saturation=0.1, hue=0.05): handles lighting variation
- RandomRotation (degrees=10): adds small geometric variation

The augmentation choices are deliberately mild and motivated by the task. Transforms that destroy high-frequency image structure (Gaussian blur, aggressive cropping) are avoided, because GAN artifacts often manifest precisely in fine texture and frequency details. Augmenting those signals away would teach the model to be invariant to the very patterns it needs to detect.

The validation loader is unchanged across CNN-A, CNN-B, and CNN-C, ensuring that any performance differences are attributable solely to architectural or pipeline changes.

5.5 Training Configuration

All three CNNs share identical training conditions: Adam optimizer (lr=0.01), BCEWithLogitsLoss, batch size 64, 30 epochs, dropout 0.5. This consistency is essential for attributing performance differences to architecture rather than training setup.

5.6 Results and Analysis

Model	Accuracy (Val)	ROC-AUC (Val)	Parameters
CNN-A	95.06%	0.99	8,482,817
CNN-B	96.18%	0.99	389,633
CNN-C	93.76%	0.98	389,633
FC Baseline (Stage 1, ref.)	81.54%	0.90	170,497

Table 2. Stage 2 validation results. Stage 1 FC included for reference.

All three CNNs substantially outperform the Stage 1 baseline, with accuracy improvements of 12 to 15 percentage points. The bottleneck in Stage 1 was the features, not the classifier, spatial structure in raw pixels carries far more discriminative information than the 201-dimensional hand-crafted vector.

CNN-B is the strongest custom architecture at 96.18% accuracy and 0.99 AUC, achieved with 22 times fewer parameters than CNN-A. GAP forces the network to produce globally meaningful feature maps rather than memorising spatial positions, resulting in better generalisation despite a dramatically smaller classifier head.

Augmentation hurt rather than helped (CNN-C vs CNN-B). CNN-C underperforms on every metric. Deepfake artifacts are predominantly high-frequency; ColorJitter and rotation directly alter the texture signals the model needs to detect. Additionally, CNN-C's highly unstable validation curves suggest 30 epochs was insufficient for convergence under augmentation, which effectively diversifies the training distribution and slows learning.

6. Stage 3: Transfer Learning with Pretrained Models

6.1 Motivation and Strategy

Stage 3 fine-tunes two pretrained ImageNet models to establish an upper bound on performance and quantify the benefit of transfer learning relative to the best from-scratch CNN. ResNet-18 and EfficientNet-B0 cover different architectural families: ResNet uses residual (skip) connections with a conventional layer structure, while EfficientNet applies compound scaling (jointly optimising network depth, width, and input resolution) to achieve better accuracy per parameter.

Full fine-tuning is used for both models: pretrained ImageNet weights are loaded, the final classification head is replaced with a single logit output, and the entire network is trained end-to-end on the deepfake dataset. This allows the pretrained representations to be adapted to the task rather than used as fixed feature extractors.

6.2 Input and Normalisation

Images are resized to 224*224, the standard ImageNet input resolution. Normalisation uses fixed ImageNet constants (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]) rather than dataset-computed statistics. This is a hard requirement: the pretrained backbone's convolutional filters were optimised against inputs in this specific distribution. Using dataset-computed statistics would feed the backbone out-of-distribution inputs, degrading the pretrained representations before fine-tuning even begins.

6.3 Augmentation

Mild augmentation is applied during training: RandomHorizontalFlip and very light ColorJitter (brightness=0.1, contrast=0.1, saturation=0.05, hue=0.02) only. Rotation is excluded. Given CNN-C's result in Stage 2 (where augmentation including rotation measurably hurt performance) a conservative approach is taken. The augmentation is present primarily to prevent the model from over-relying on exact pixel values, not to diversify the training distribution.

6.4 Training Configuration

Both pretrained models are trained with Adam (lr=0.0001), BCEWithLogitsLoss, batch size 32, and 20 epochs. The learning rate is set lower than Stage 2 (0.0001 vs 0.01). The pretrained backbone weights are already in a well-initialised region of the loss landscape, and an aggressive learning rate would update them too forcefully, disrupting the pretrained representations rather than gently adapting them.

A ReduceLROnPlateau scheduler is introduced: if validation loss does not improve for 3 consecutive epochs, the learning rate is halved. This is standard practice for fine-tuning workflows, allowing the optimiser to make progressively finer adjustments as the model converges. The scheduler state is saved in the checkpoint so that patience counters are correctly restored on reload.

6.5 Model Architectures

ResNet-18 (approximately 11.2M parameters): The original final fully connected layer (`Linear(512, 1000)`) is replaced with `Linear(512, 1)`. The input feature size is read from `backbone.fc.in_features` rather than hardcoded, making the code robust to architecture variants. All other layers retain pretrained weights and remain fully trainable.

EfficientNet-B0 (approximately 4.01M parameters): The original classifier is `Sequential(Dropout(0.2), Linear(1280, 1000))`. It is replaced with `Sequential(Dropout(0.2), Linear(1280, 1))`, preserving the original Dropout since it was part of the backbone's design. EfficientNet-B0 has less than half the parameters of ResNet-18, yet achieves better performance on this task due to its compound scaling design.

6.6 Results and Analysis

Model	Accuracy (Val)	ROC-AUC (Val)	Parameters
ResNet-18	95.56%	0.9895	11,177,025
EfficientNet-B0	96.90%	0.9958	4,008,829
CNN-B (Stage 2, ref.)	96.18%	0.9939	389,633

Table 3. Stage 3 validation results. CNN-B included for reference.

EfficientNet-B0 achieves the strongest result in Stage 3 at 96.90% accuracy and 0.9958 AUC. An initial run of ResNet-18 used a higher experimental learning rate, at which the pretrained representations showed signs of disruption and training loss diverged from validation loss after epoch 12. After correcting to $\text{lr}=0.0001$ the training curves showed slight improvement, though some instability persisted. Stage 3 results are reported from the corrected run.

This experience illustrates a practical distinction between fine-tuning and training from scratch: fine-tuning is more sensitive to learning rate because the weights are already in a useful region of the loss landscape, and overshooting that region is easier than reaching it.

7. Final Test Set Evaluation

The test set is evaluated exactly once, after all models across all three stages are finalised. Three separate test loaders are constructed to match each stage's required input format: Stage 1 uses

pre-extracted 201-dim feature vectors; Stage 2 uses 128*128 images with dataset normalisation; Stage 3 uses 224*224 images with ImageNet normalisation. Each model receives exactly the input format it was trained on.

Model	Accuracy	F1-Score	ROC-AUC	Parameters
FC Baseline (Stage 1)	81.64%	81.55%	0.8945	170,497
CNN-A (Stage 2)	94.98%	94.96%	0.9898	8,482,817
CNN-B (Stage 2)	95.94%	95.93%	0.9921	389,633
CNN-C (Stage 2)	93.14%	93.17%	0.9826	389,633
ResNet-18 (Stage 3)	95.24%	95.23%	0.9891	11,177,025
EfficientNet-B0 (Stage 3)	97.22%	97.20%	0.9969	4,008,829

Table 4. Final test set results across all six models.

The validation-to-test gaps are uniformly small across all models, with the biggest drop being CNN-C at 0.62 percentage points. This consistency confirms that the validation set was a reliable proxy for test performance throughout development, and that no architecture or hyperparameter decisions were inadvertently overfit to the validation distribution.

7.1 Stage Progression

Stage 1 to Stage 2 (+13%) is the largest jump in the project. The FC baseline was limited by its 201-dimensional feature vector, not by the classifier. CNNs access the full spatial structure of the image (relationships between regions, artifact texture, boundary characteristics) and this is information that no fixed hand-crafted descriptor can fully encode.

Stage 2 to Stage 3 (+1 to 2%) is real but modest. EfficientNet-B0 improves on CNN-B's 95.94% to reach 97.22%, but the gap is far smaller than the Stage 1 to Stage 2 jump. The custom CNNs, particularly CNN-B, are already highly competitive. Pretrained weights are a strong prior, not a guarantee of superior performance.

7.2 Notable Results

EfficientNet-B0 achieves the best result across all stages at 97.22% accuracy and 0.9969 AUC, with only 4.01M parameters. Its compound scaling design proved well-suited to this task, delivering the strongest generalisation on the held-out test set.

CNN-B performs similar to ResNet-18 on the test set (95.94% vs 95.24%). A 389,633-parameter network trained from scratch outperforms an 11.2M parameter pretrained model. This

demonstrates that pretrained weights are a strong prior but not a substitute for appropriate regularisation. CNN-B's GAP architecture provided structural regularisation throughout training, while ResNet-18's larger capacity and the instability observed in its training curves limited its final generalisation despite the advantage of pretrained representations.

CNN-C remains the weakest custom CNN at 93.14%, consistently below even CNN-A. The augmentation findings from validation held up on the unseen test set, reinforcing the conclusion that the augmentation strategy eroded rather than strengthened the model's ability to detect GAN-specific artifacts.

8. Cross-Distribution Generalisation

All six trained models are applied to the Stable Diffusion dataset, a set of synthetically generated images produced by a fundamentally different generation method than the GAN-based fakes in the training data. Since all images in this set are fake, every image classified as real is a missed detection. The experiment tests whether the models learned general deepfake detection or GAN-specific recognition.

Every model, regardless of architecture or capacity, failed almost entirely to detect the Stable Diffusion images as fake. Detection rates were effectively zero across all six models, with images classified as real at high confidence.

Model	Detected as Fake	Detection Rate
FC Baseline	25 / 3,000	0.8%
CNN-A	3 / 3,000	0.1%
CNN-B	40 / 3,000	1.3%
CNN-C	50 / 3,000	1.7%
ResNet-18	2 / 3,000	0.1%
EfficientNet-B0	54 / 3,000	1.8%

Table 5. Cross-distribution detection rates on 3,000 Stable Diffusion images (all ground truth = fake).

8.1 Interpretation

The result has a clear explanation. Every model was trained exclusively on GAN-generated fakes, which exhibit a characteristic set of artifacts: frequency patterns from the adversarial

training process, texture inconsistencies at fine scales, and blending artifacts at boundaries. All six models converged on these dataset-specific signals.

Stable Diffusion images have an entirely different artifact profile, rooted in iterative diffusion-based denoising from Gaussian noise and producing images with fundamentally different frequency and texture characteristics. The models encountered nothing familiar in these images and defaulted to predicting real.

Three conclusions follow. First, the 97.22% test accuracy should be understood as GAN recognition performance within a single generation-method distribution, not a general deepfake detection capability. Second, architecture did not matter: the FC baseline and EfficientNet-B0 failed equally, ruling out model capacity or feature quality as the bottleneck. Third, this is a data problem rather than a modelling problem. A robust detector requires training data covering diverse generation methods (GANs, diffusion models, VAEs) alongside augmentation strategies that simulate cross-domain variation. No architectural improvement within the current training distribution would address this failure.

9. Discussion

The project followed a structured, controlled experimental framework that isolated one variable at a time: feature representation in Stage 1, architectural design in Stage 2, and pretrained representations in Stage 3. This design makes results interpretable, as performance differences can be attributed to specific choices rather than confounded by multiple simultaneous changes.

The most instructive findings are the unexpected ones. CNN-B outperforming CNN-A with 22 times fewer parameters demonstrates that architectural decisions (specifically the pooling strategy) can matter more than raw parameter count. CNN-C underperforming CNN-B challenges the common assumption that data augmentation is universally beneficial. In tasks where the discriminative signal is texture and frequency-based, augmentation can erode the very signal being learned. CNN-B outperforming ResNet-18 on the test set challenges the assumption that pretrained models automatically dominate from-scratch models at this scale and dataset size.

The honest limitation of all results is their within-distribution nature. A 97% accuracy figure is only meaningful relative to the distribution it was measured on. The cross-distribution experiment exposes this directly: every model, including EfficientNet-B0, fails completely when the generation method changes. This is the defining open problem in deepfake detection, namely building models that generalise across generation architectures rather than within them.

10. Conclusion

This project built and evaluated a complete deepfake image detection system using Deep Learning, progressing from a hand-crafted feature baseline through custom CNNs to fine-tuned pretrained models. The key findings are:

- The largest performance gain came from replacing hand-crafted features with CNN spatial learning. Engineered descriptors are the limiting factor in classical pipelines, not the classifier.
- Global Average Pooling (CNN-B) outperformed the flatten-head CNN-A and the augmented CNN-C, demonstrating that classifier head design and pooling strategy have significant impact on both parameter efficiency and generalisation.
- EfficientNet-B0 achieved the best overall result (97.22% accuracy, 0.99 AUC), though the improvement over the best from-scratch CNN was modest, and was achieved with only 4.01M parameters.
- All models failed completely on Stable Diffusion-generated images, revealing that high within-distribution accuracy does not imply general deepfake detection capability. The models learned GAN fingerprints, not fakeness.

Meaningful directions for future work include: training on multi-source datasets covering diverse generation methods to improve cross-distribution robustness and applying frequency-domain augmentation strategies that better simulate cross-distribution variation without eroding texture-level artifacts.

Sources:

[Generative Adversarial Networks \(2020\), Goodfellow et al.](#)

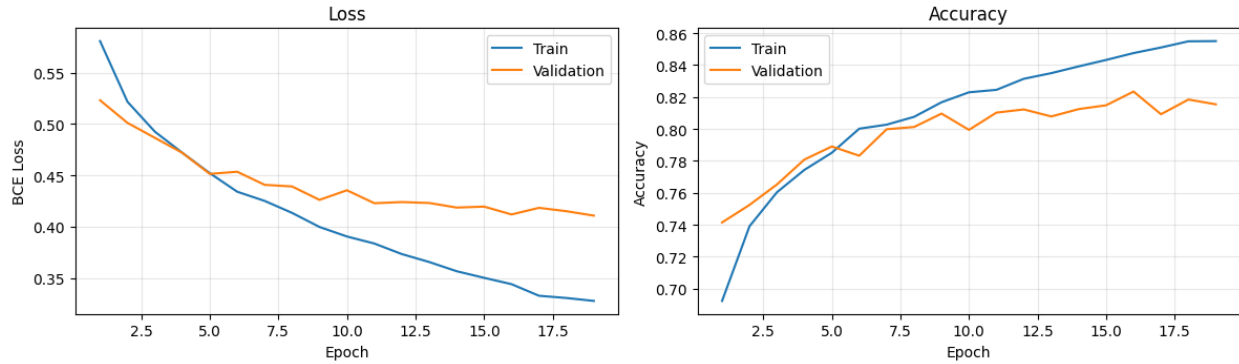
[Deep Residual Learning for Image Recognition \(2016\), He et al.](#)

[Network In Network \(2013\), Lin et al.](#)

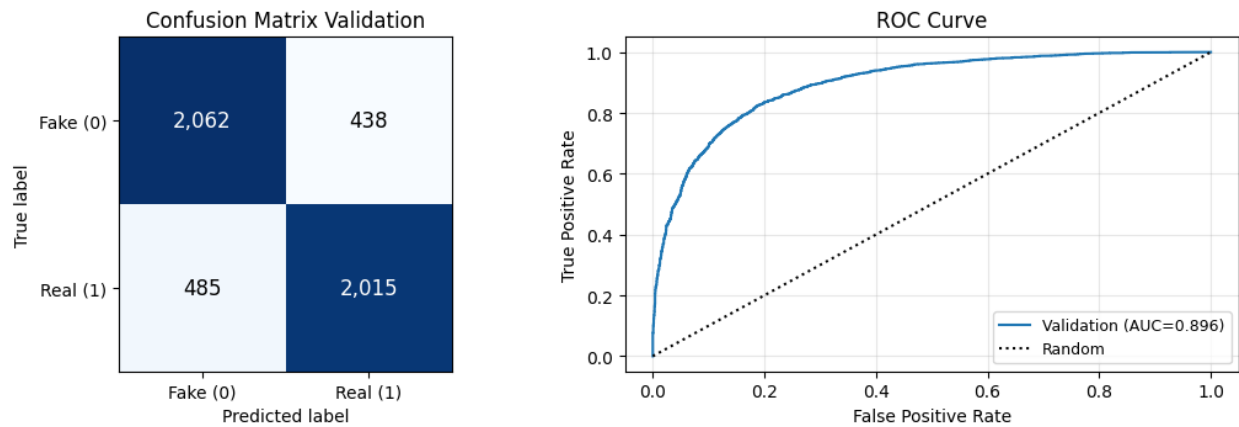
Appendix:

FC:

FC Network Training Curves

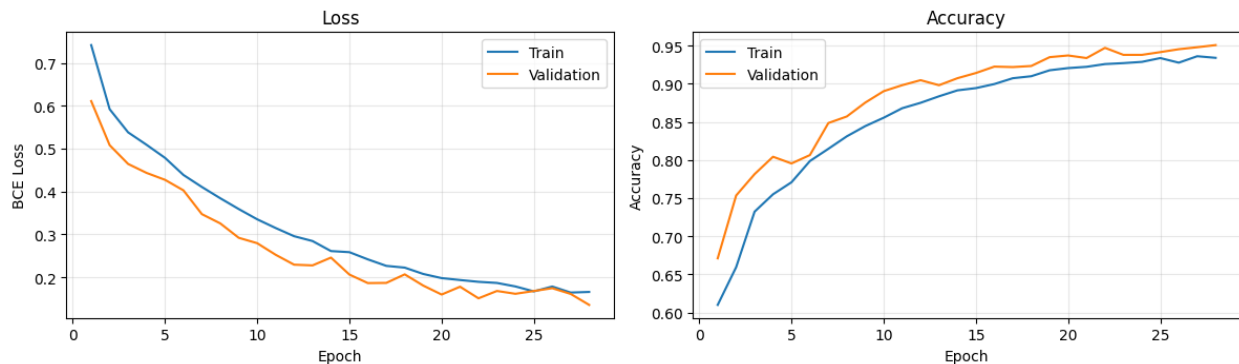


FC Network Evaluation

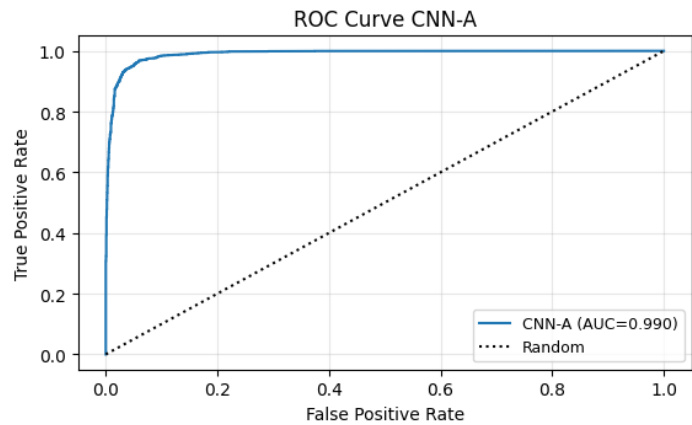
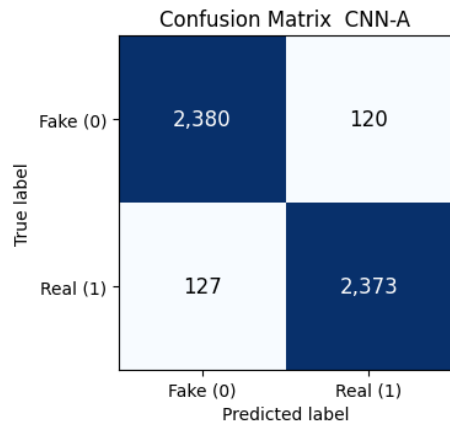


CNN-A:

CNN-A Training Curves

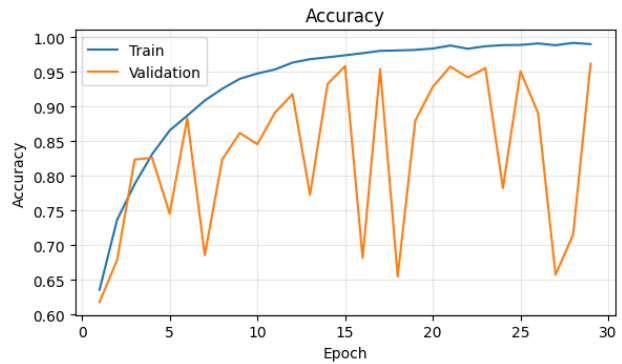
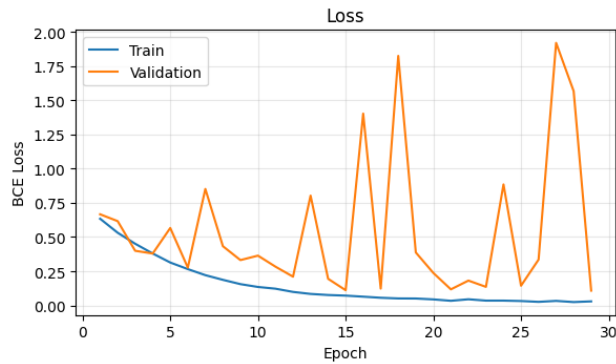


CNN-A Evaluation

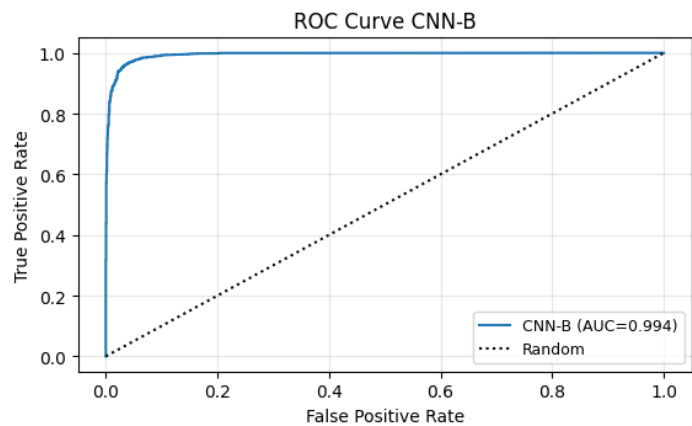
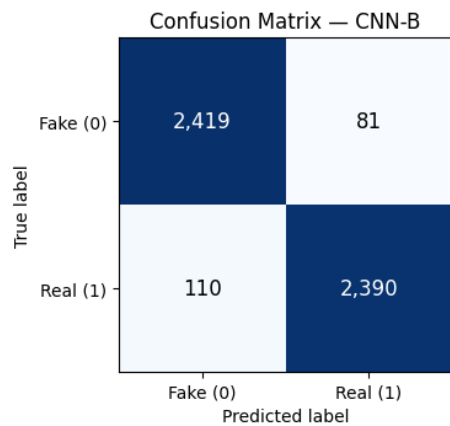


CNN-B:

CNN-B Training Curves

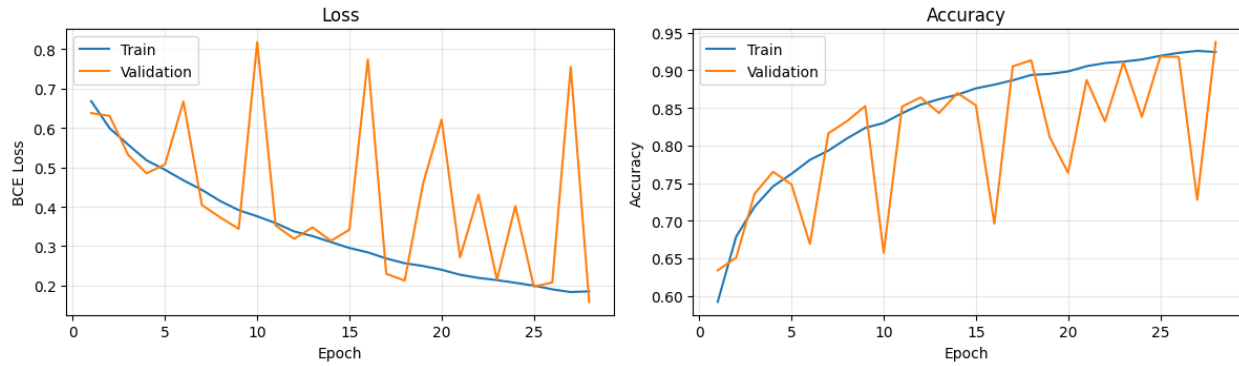


CNN-B Evaluation

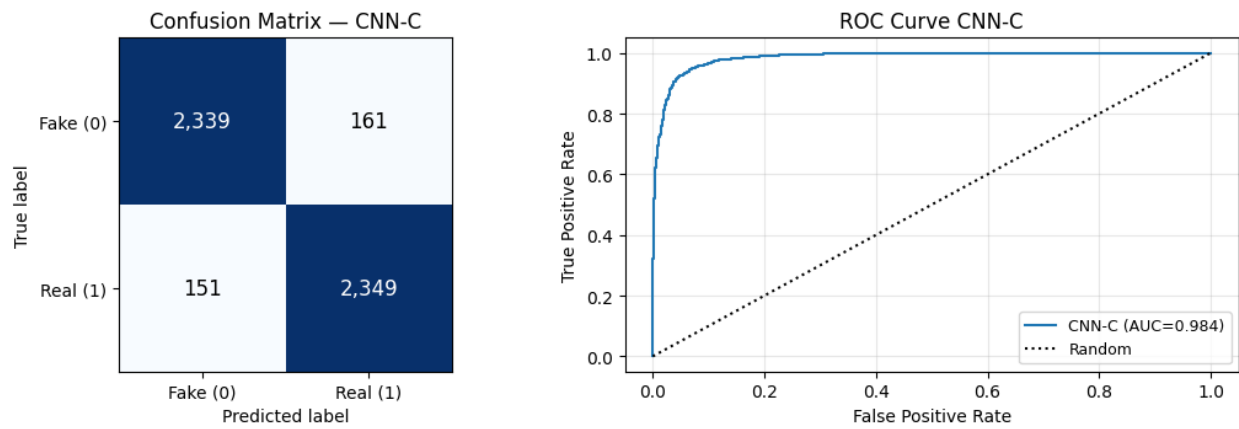


CNN-C:

CNN-C Training Curves

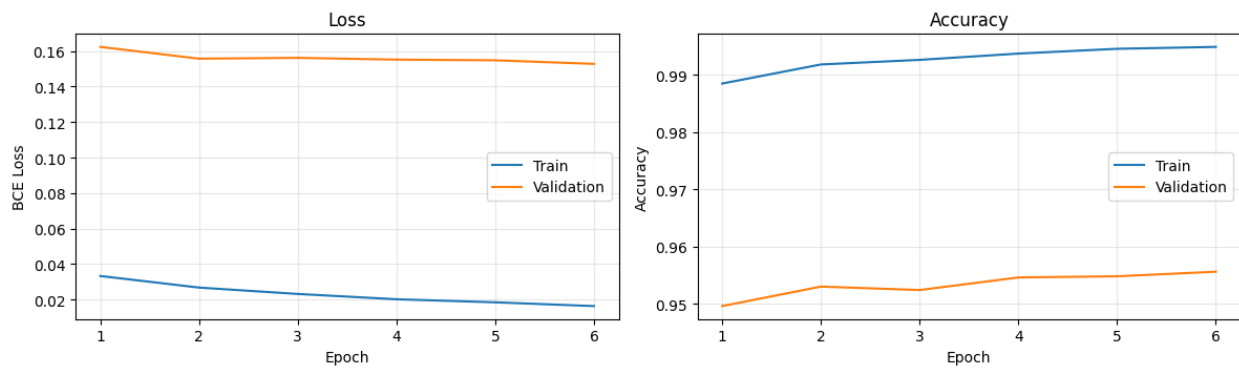


CNN-C Evaluation

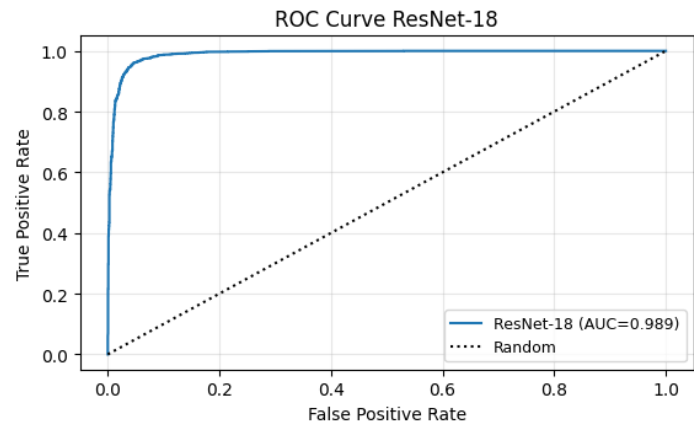
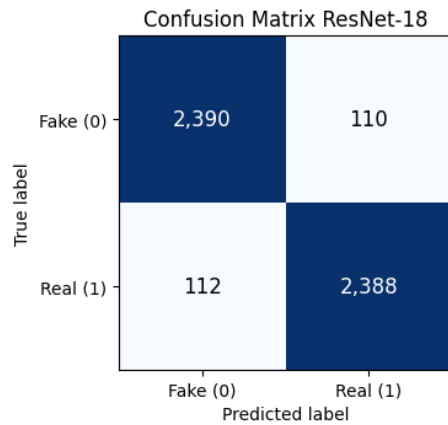


ResNet-18:

ResNet-18 Training Curves

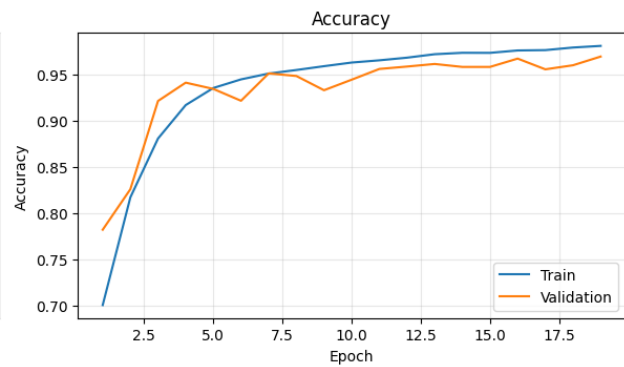
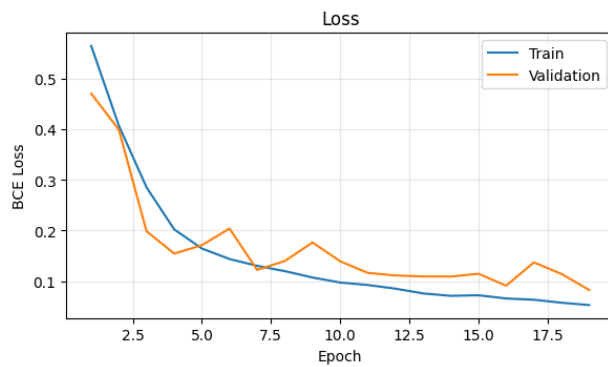


ResNet-18 Evaluation

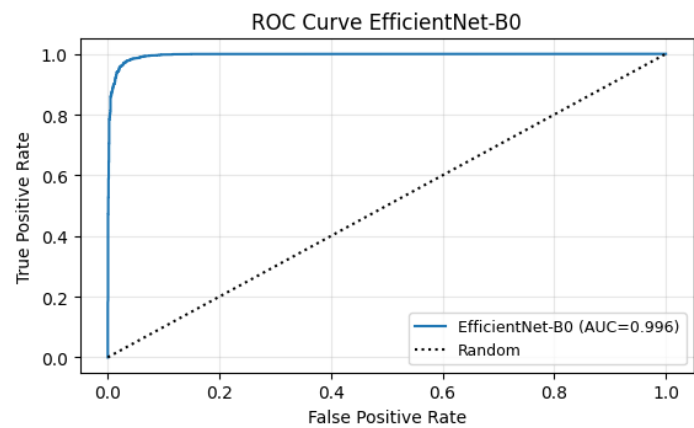
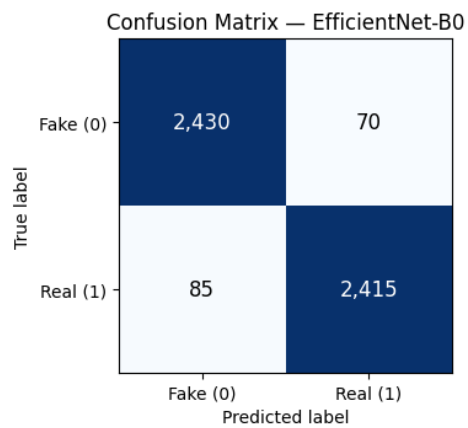


EfficientNet-B0:

EfficientNet-B0 Training Curves



EfficientNet-B0 Evaluation



Final Test Set ROC Curves:

